

AI LA x Qualcomm Robot Soccer Cup: Developer Journey Guide

This technical guide provides a comprehensive workflow for developers participating in the Robot Soccer Cup, organized by AI LA and Qualcomm.

[Overview](#)

[The Tournament: 1-on-1 World Cup Style](#)

[Redemption Cup](#)

[Game Rules](#)

[Schedule](#)

[Logistics](#)

[Journey at a glance](#)

[Prepare the kit and workspace](#)

[What you need](#)

[Safety before software](#)

[Pre-run checkpoint](#)

[Core Architectural Concepts](#)

[Control and data flow](#)

[UNO Q Hardware Map](#)

[Bring up the UNO Q baseline](#)

[Build and run](#)

[Use the camera BOOT button](#)

[Baseline checkpoint](#)

[Build your first robot behaviour](#)

[Named motion commands](#)

[Actuators and feedback](#)

[Read sensors and make decisions](#)

[Design a bounded policy](#)

[Bring up the ESP-32-S3 camera](#)

[One-time Arduino IDE setup \(Already done\)](#)

[Test the stream](#)

[Camera and team state checkpoint](#)

[Add the AI soccer layer](#)

[Recommended application loop](#)

[Use the AI agent](#)

[Model import](#)

[1. Upload Captured Images](#)

[2. Label Bounding Boxes](#)

[3. Configure the FOMO Impulse](#)

[4. Image: Feature Generation](#)

[5. Object Detection: Neural Network Settings](#)

[6. Deployment](#)

[Wall / Field-Side Detection](#)

[Concept](#)

[HSV Ranges for the Field Tape](#)

[Camera Stream](#)

[Suggested Module Layout](#)

[Expected Console Output](#)

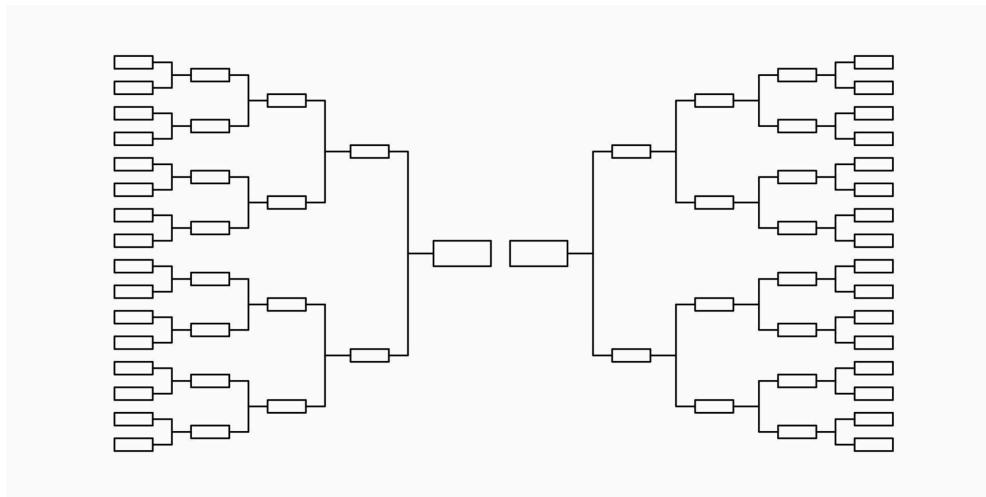
[Tuning Tips](#)

[Troubleshooting](#)

Overview

The Tournament: 1-on-1 World Cup Style

- Format: 1 robot vs 1 robot, Round of 32 with single elimination.



- Duration: 5-minute lightning matches.
- Tie-Breakers: If the score is level, ball touches will determine the winner.
- Prizes (each team member):
 - Winner: LEGO FIFA World Cup Official Trophy 43020
 - Runner-up: LEGO Soccer Ball 43019

Redemption Cup

As teams are knocked out of the tournament, they can compete for one additional challenge and win other prizes:

1. Best Goal Celebration

- Robot scores and then autonomously performs a celebration.
- Recommend use of LEDs, movement patterns, or other creative behaviors.
- Prize (each team member): Govee Smart Light Bars

2. Trick Shot Challenge

- Robot must score while navigating around obstacles.
- Prize (each team member): LEGO Trickshot 43021

3. Fastest Ball Detection

- Measured by the time between challenge start and the robot initiating movement toward the detected ball.
- *Prize (each team member): Nike x LEGO Soccer Ball 5010262*

4. Precision Course

- Robot follows a taped course using its onboard sensor.
- Winner determined by speed and accuracy.
- *Prize (each team member): Fanttik E1 Max Precision Screwdriver*

5. Attendees' Choice

- Community-selected challenge with judging determined by participants.
- *Prize (each team member): LEGO Umbreon vs Garchomp Championship Battle 72165*

Game Rules

Each game will be 5 minutes in length. The team with the most goals scored wins the game. If tied, the team with the most ball touches (kicks) wins the game.

Each team will be designated as either a red team or a blue team for each game. The object is to score a goal in the opponent's goal. E.g. if you are on the red team, you will be aiming to score a goal in the blue goal. We will not switch sides during the game, but your team color may change during each round of play.

Each game will start with one opponent on each side of the field. Your robot's chassis will need to be behind the black line on your side of the field (starting position). The ball will be placed on the yellow dot in the middle of the field. Once your game's referee starts the timer, you will turn your robot on and the game will commence.

Penalty Card	Action
Yellow Card	Cornering opponent's robot in a wall.
	Tipping over the opponent's robot.
Red Card	2 Yellow Cards

If a team acquires a yellow card, the game will restart with each team back to the starting position. Once both robots are ready, the game will resume.

Schedule

Timing	Activity
9 AM	Check-in and breakfast
10 AM	Event kickoff and Quick Edge Impulse workshop
11 AM	Development begins

Timing	Activity
4 PM	Tournament Kick-off and Redemption Cup thereafter
6 PM	Winner announcement and networking

Logistics

- Meals, coffee, and refreshments will be provided throughout the event
- Teams may take their kit for the day as long as the Team Lead has signed and returned the Loaner Agreement.

Journey at a glance

Treat the robot as a layered system. Prove each layer before adding the next, and keep a stop condition in every behavior that can move hardware.

Stage	Focus	Developer action	Exit checkpoint
0	Safety	Inspect, charge, wire, and raise the wheels for first motion	Robot is mechanically safe to power
1	Firmware	Build the UNO Q sketch and verify Bridge health	MCU, serial, and Bridge respond
2	Control	Run short, timed moves through MiniAutoRobot	Motion can start, interrupt, and stop
3	Sensing	Read line, ultrasonic, battery, program, and team state	Policy inputs are valid and bounded
4	Vision	Flash and connect to the camera MJPEG access point	Frames arrive at the documented endpoint
5	AI policy	Convert detections into short, guarded actions	No/stale/ambiguous input produces stop
6	Match readiness	Validate recovery, team toggle, and final shutdown	Team can operate the robot predictably

Prepare the kit and workspace

What you need

- **[Provided] Robot hardware:** [Arduino UNO Q](#); pre-assembled [Hiwonder miniAuto](#) chassis and connected motor/sensor board; ESP32-S3 vision module; glowing ultrasonic sensor; four-channel line sensor; charged 18650 cells and holder.

- **Development tools:** [Arduino App Lab](#) for the UNO Q application. [Arduino IDE](#) is required only when the camera firmware must be configured or reflashed.
- **Repository:** <https://github.com/aaishikasb/qc-robot-soccer-cup>; the unpacked files in this project. Treat miniAutoDriver.zip as a distribution snapshot, not the editable source of truth.
- **Python runtime:** the UNO Q App Lab environment supplies `arduino.app_utils`. The imported robot driver does not require third-party Python packages.

Safety before software

- Keep the wheels raised and clear of the bench during first motion tests.
- Confirm mecanum-wheel orientation, cable routing, sensor connections, battery polarity, and the absence of loose conductive hardware before applying power.
- Use the supplied charger or a Hiwonder-approved alternative. Stop using damaged, swollen, wet, unusually hot, or abnormal cells or chargers.
- Keep a physical power disconnect within reach. Begin with low speed and short, nonzero durations.
- Do not push the powered robot by hand; the vendor warns that forced movement can damage the drive system or electronics.

Pre-run checkpoint

- Robot is mechanically complete and wiring has been checked against the kit revision.
- Battery is charged, correctly installed, and switched off until software is ready.
- Wheels can spin without touching the bench, hands, cables, or tools.
- App Lab can open the repository and sees `app.yaml` plus the default sketch profile.

Core Architectural Concepts

The project intentionally separates perception transport, hard real-time hardware control, and high-level behavior. Keep those boundaries intact as your AI application grows.

Layer	Source	Owns
ESP32-S3 camera controller	camera/HiwonderCamStream/HiwonderCamStream.ino	Camera capture, Wi-Fi access point, MJPEG server, BOOT-button state over I2C 0x79
UNO Q microcontroller	sketch/sketch.ino	Motors, mecanum mixing, timed auto-stop, servo, buzzer, LEDs, sensors, serial commands, Bridge providers

Layer	Source	Owns
UNO Q Linux / Python	python/robot_client.py and python/main.py	Bridge serialization, program guards, application routine, future inference and soccer policy

Control and data flow

- Camera frames -> <http://192.168.5.1:81/stream> -> Python inference/policy
- BOOT button -> ESP32-S3 I2C 0x79 -> UNO Q program/team state
- Python action -> Arduino_RouterBridge -> UNO Q motor/actuator driver
- UNO Q sensors -> Bridge JSON -> Python policy
- Keep inference, model loading, and soccer strategy on the Linux/Python side.
- Keep direct I/O, motor mixing, clamping, and fail-safe timing on the microcontroller side.
- The camera stream is independent of the movement fail-safe. Loss of frames must still make the Python policy stop the robot.

UNO Q Hardware Map

Device	UNO Q pin or address
Motor PWM M0, M1, M2, M3	D10, D9, D6, D11
Motor direction M0, M1, M2, M3	D12, D8, D7, D13
Onboard WS2812 RGB	D2
Passive buzzer	D3
Battery divider	A3
Glowing ultrasonic sensor	I2C 0x77
Four-channel line sensor	I2C 0x78
Camera BOOT-button controller	I2C 0x79

Bring up the UNO Q baseline

Build and run

1. Import or open the aforementioned repository as an Arduino UNO Q application in App Lab.
2. Use the default profile declared in sketch/sketch.yaml: platform arduino:zephyr with Arduino_RouterBridge (0.4.2).
3. Build and upload sketch/sketch.ino to the UNO Q.

4. Start the application described by app.yaml. Its Linux entry point is python/main.py.
5. Watch the application output for the health and sensor payloads before enabling motion.

health : {"robot":"hiwonder_miniauto","mcu":"uno_q","bridge":true,"serial":true}

Expected identity. The health payload should identify hiwonder_miniauto on uno_q and report both bridge and serial as true.

Use the camera BOOT button

Action / state	Observed behavior
Idle	Ultrasonic RGB flashes blue every 1.5 seconds
Short press while idle	Red > yellow > solid green countdown, then program_enabled becomes true
Short press while running	Stops motors, disables the program, and returns to idle
Hold for 5 seconds	Toggles team state; onboard RGB is red when false and blue when true

Important current behavior. The stop Bridge RPC stops all motors and also sets program_enabled to false. Inside an active routine, rely on nonzero drive durations for ordinary motion completion; use stop() as the session-ending fail-safe and finally cleanup.

Baseline checkpoint

- Health reports bridge=true and serial=true.
- Sensor JSON parses and includes program_enabled plus hold_toggle.
- A short BOOT press produces the red/yellow/green enable sequence.
- A second short press interrupts movement and returns the robot to idle.
- A five-second hold changes the team color without starting motion.

Build your first robot behaviour

Start in python/main.py and reuse MiniAutoRobot. The wrapper centralizes Bridge calls, program-state checks, routine interruption, and shutdown behavior.

- Use short, nonzero durations. Firmware clamps motion duration to 0–5000 ms; 0 means continuous motion and requires an explicit stop.
- MiniAutoRobot.drive waits for the timed move to finish and checks program state again, so a BOOT-button stop can interrupt a routine.
- MiniAutoRobot.run_program runs the routine once for each enabled session and always stops in its cleanup path.

- Keep `robot.stop()` in a finally block for every application entry point that can move hardware.

Named motion commands

Motion	Accepted values
Forward	forward, f
Backward	backward, back, reverse, b
Strafe left	left, strafe_left, a
Strafe right	right, strafe_right, d
Rotate left	rotate_left, turn_left, q
Rotate right	rotate_right, turn_right, e
Stop	stop, x

Actuators and feedback

MiniAutoRobot method	Behavior
<code>buzz()</code>	Plays a three-part chirp
<code>led(on)</code>	Sets onboard and ultrasonic RGB lights to white or off
<code>drive_raw(m0, m1, m2, m3, ms)</code>	Sets per-motor values clamped to -255..255 with optional auto-stop
<code>health()</code>	Returns robot, MCU, Bridge, and serial identity/status

Read sensors and make decisions

Field	How to interpret it
<code>line_ok</code>	false when the line-sensor I2C read fails; line bits are then zeroed
<code>line_digital / trace_digital</code>	The same four compatibility bits
<code>ultrasonic_mm / ultrasonic_cm</code>	-1 when the ultrasonic I2C read fails
<code>battery_mv</code>	Estimate from the A3 divider calibration; not a precision fuel gauge
<code>program_enabled</code>	Whether guarded user motion/servo calls may proceed
<code>hold_toggle</code>	Team selector: false/red, true/blue

Field	How to interpret it
ir	Reserved compatibility field; always -1 in this driver

Design a bounded policy

- Validate every sensor value before acting. Treat -1 distances and line_ok=false as unavailable input, not as a valid reading.
- Prefer a short action followed by a fresh observation instead of long open-loop movement.
- Stop when the program is disabled, a required sensor is invalid, a condition times out, or an exception occurs.
- Keep motor and servo limits in firmware even if Python validates them too.

```
def decide(sensors):
    if not sensors.get("program_enabled"):
        return None
    distance = sensors.get("ultrasonic_mm", -1)
    if distance <= 0:
        return None          # unavailable -> safe stop
    if distance < 400:
        return ("rotate_left", 90, 250)
    return ("forward", 90, 250)
```

The sketch also contains a simple serial-controlled obstacle-avoidance demo. It checks every 100 ms, rotates left below 400 mm, and otherwise drives forward. Because a failed/nonpositive distance follows its forward branch, supervise that demo closely and use an explicit safe default in participant policies.

Bring up the ESP-32-S3 camera

The Hiwonder ESP32S3-CAM V1.0 firmware creates its own Wi-Fi access point and exposes the GC2145 camera as a QVGA MJPEG stream. This controller is flashed separately from the UNO Q.

One-time Arduino IDE setup (Already done)

1. Install esp32 by Espressif Systems version 2.0.11 in Boards Manager. Repository preparation notes report GC2145 initialization failures with core 3.x.
2. Open camera/HiwonderCamStream/HiwonderCamStream.ino.
3. Assign each robot a unique CAMERA_SSID. The checked-in value is miniAuto_CAM_01.
4. Set a password of at least eight characters. The checked-in value is Q01pass!.
5. Upload with the settings below, confirm the verified hash/hard reset, then fully power-cycle the camera.

Arduino IDE setting	Value
Board	ESP32S3 Dev Module
PSRAM	OPI PSRAM
Flash Size	8MB (64Mb)
Partition Scheme	Huge APP (3MB No OTA/1MB SPIFFS)
USB CDC On Boot	Disabled
Flash Mode	DIO
Upload Speed	921600
Port	Serial/COM port assigned to the camera

Test the stream

1. Join the camera Wi-Fi network using the configured SSID and password.
2. Open <http://192.168.5.1:81/> and select Start Stream.
3. For an application consumer, use the direct endpoint <http://192.168.5.1:81/stream>.

`CAMERA_URL = "http://192.168.5.1:81/stream"`

The access point uses 192.168.5.1/24 and port 81. The camera captures 320×240 RGB565 frames, converts them to JPEG at quality 80, and serves multipart MJPEG with two PSRAM frame buffers. Software JPEG conversion can make the stream choppy; stay near the access point, disconnect unused clients, and keep QVGA while diagnosing.

Camera and team state checkpoint

- The index page loads and the direct MJPEG endpoint produces frames.
- Every robot has a unique SSID; fleet naming can use `miniAuto_CAM_01`, `_02`, `_03`, and so on.
- A short BOOT press reaches the UNO Q as a program toggle over I2C 0x79.
- A five-second hold changes `hold_toggle` and the onboard team color.

Add the AI soccer layer

The repository deliberately stops at hardware control and camera transport. Bring your own model and inference runtime, then add the policy on the UNO Q Linux/Python side.

Do not place model binaries or generated inference artifacts in the aforementioned driver repository, instead, clone or create your own fork.

Recommended application loop

Step	Responsibility
1. Acquire	Read the latest frame from the configured MJPEG endpoint; record a timestamp
2. Preprocess	Resize/normalize exactly as the participant-supplied model expects
3. Infer	Produce detections, classes, confidence, or another task-specific result
4. Validate	Reject stale frames, missing models, no detections, ambiguous detections, or invalid outputs
5. Decide	Combine perception with hold_toggle, distance, line state, and match logic
6. Act	Issue one short timed MiniAutoRobot action
7. Reobserve	Read a new frame/sensor state before the next action

Use the AI agent

The repository includes a reusable agent skill at `.agents/skills/uno-q-miniauto`. Your AI agent can use it to turn a robot idea into focused changes across the existing UNO Q firmware, Python Bridge application, camera firmware, configuration, and documentation while preserving the driver's safety behavior and public interfaces.

The skill extends the checked-out source rather than generating a separate driver stack. It chooses the smallest appropriate layer: high-level behavior normally stays in Python, direct hardware control and fail-safe timing stay in the UNO Q sketch, and camera-only changes stay in the ESP32-S3 firmware.

Model import

1. Upload Captured Images

Create an Edge Impulse project or clone our project with prelabelled capture data:
<https://studio.edgeimpulse.com/public/1085406/live>

2. Label Bounding Boxes

For object detection training:

- Label `soccerball` objects with bounding boxes.
- Label `robot` objects with bounding boxes.
- Leave `goal` objects with bounding boxes.

Keep bounding boxes tight around the visible object. Consistent labeling usually improves FOMO training results. We have prelabelled a capture dataset for you.

3. Configure the FOMO Impulse

Input Block

- Image size: 96 x 96
- Color depth: RGB
- Resize mode: squash
- Train on data subset: ****100%****

Learning Block

- Model: Object Detection (Images)
- Input features: check the Image box
- Output features: 3 (goal, robot, soccer_ball)

Adjust these settings if your dataset, target runtime, or accuracy requirements differ.

4. Image: Feature Generation

After training, review the validation metrics and class behaviour. Check that each class is detected reliably before exporting.

5. Object Detection: Neural Network Settings

Training Settings

- Training cycles: 150-180 as a starting range
- Learning rate: 0.001
- Training processor: CPU

After the model performs well enough for live testing, export it as an Edge Impulse .eim file for Linux aarch64.

The .eim file is generated from your own Edge Impulse project and is not included in the repository by default.

6. Deployment

The Deployment Target is the Arduino UNO Q, hit build and download the .eim file for use in your application.

Wall / Field-Side Detection

The soccer field has coloured tape strips on the walls — red tape on one side, blue tape on the other. Your robot can use the camera stream to detect which wall it is currently facing and determine whether it is on its own side or the opponent's side.

This is a good challenge to implement using HSV colour classification — no ML model or training data required.

Concept

HSV (Hue, Saturation, Value) colour space makes it straightforward to isolate a specific colour regardless of brightness. The approach:

1. Grab a frame from the MJPEG camera stream using OpenCV.
2. Convert the BGR frame to HSV with `cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`.
3. Create a mask for red pixels and a mask for blue pixels using `cv2.inRange` with calibrated hue/saturation/value bounds.
4. Count the masked pixels for each colour and express them as a fraction of the total frame.
5. If either colour exceeds a minimum coverage threshold (~2 % of the frame is a good starting point), report it as the dominant side.
6. Compare the detected wall colour against the robot's active team (from `robot.hold_toggle()`) to determine OWN SIDE or OPPONENT SIDE

HSV Ranges for the Field Tape

OpenCV uses Hue 0–179, Saturation 0–255, Value 0–255. Red wraps around 0/180 in HSV so it needs two ranges:

Colour	Hue range / notes
Red	Hue 0–10 and 160–179 (combine two <code>inRange</code> masks with <code>bitwise_or</code>)
Blue	Hue 100–130 (single <code>inRange</code> mask)

Start with Saturation ≥ 120 and Value ≥ 70 to filter out dark or washed-out pixels. Adjust based on your lighting conditions.

Camera Stream

The ESP32-S3 camera serves an MJPEG stream at:

`http://192.168.5.1:81/stream`

Open it with `cv2.VideoCapture(url)`. The stream needs a short warmup — discard frames in a loop until a non-empty frame arrives before running detection.

Suggested Module Layout

Keep camera setup (URL, `VideoCapture`, warmup) in `main.py` and pass the open capture object into a dedicated detector class or function. This keeps colour logic reusable and testable independently of the robot loop.

Expected Console Output

A well-implemented detector should print something like this each loop iteration:

[FIELD] team=RED wall=RED -> OWN SIDE
[FIELD] team=RED wall=BLUE -> OPPONENT SIDE
[FIELD] team=RED wall=UNKNOWN -> UNKNOWN

Adding a per-frame diagnostic line showing raw red/blue coverage percentages is useful for tuning thresholds:

[WallDetector] red=5.22% blue=0.01% side=RED

Tuning Tips

- If the detector reports UNKNOWN when tape is clearly visible, lower the minimum coverage threshold or widen the hue range.
- If it assesses false-positives on robot jerseys or the floor, raise the saturation minimum or narrow the hue range.
- The black textured wall can register as very dark blue — raising the value minimum (e.g. ≥ 70) helps exclude it.
- Print coverage percentages for a few seconds at startup to calibrate thresholds for your specific lighting.

Troubleshooting

Symptom	What to check
Python cannot control robot	Confirm arduino:zephyr, Arduino_RouterBridge 0.4.2, can upgrade to 0.4.3, Bridge health, and BOOT-enabled program state
Guarded call stops immediately	Check program_enabled; use a short BOOT press and wait for solid green
Motor mapping is wrong	Raise wheels; run serial 1–4 one channel at a time; verify the physical kit revision before changing pins/mixing
Sensor JSON shows -1 / false	Treat input as unavailable; inspect I2C wiring and addresses 0x77/0x78; do not drive from invalid data
Camera initialization fails	Use Espressif core 2.0.11 and the exact board, PSRAM, flash, partition, USB CDC, and DIO settings; power-cycle
Camera port is missing	Try a data-capable cable/another port; on Windows, install the appropriate CH34x driver if required

Symptom	What to check
Stream is slow/choppy	Stay close, remove unused clients, retain QVGA; RGB565-to-JPEG conversion occurs in software
BOOT button does not reach UNO Q	Check camera firmware and I2C 0x79 wiring; firmware notes suggest swapping CAM_SDA_PIN/CAM_SCL_PIN if 0x79 is never seen